

5. ROS2 Packages

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Package Overview

Each robot may have many functions, such as motion control, visual perception, autonomous navigation, etc. If we put all the source code for these functions together, it would work, but when we want to share some of these functions with others, we find that the code is mixed together and difficult to separate.

The package mechanism follows this principle. We divide code of different functions into different packages, trying to minimize the coupling between them. When we need to share with others in the ROS community, we only need to explain how to use the package, and others can quickly start using it.

Therefore, the package mechanism is one of the important methods to improve software reusability in ROS.

2. Creating a Package

How to create a package in ROS2? You can use this command:

```
ros2 pkg create <package_name> --build-type <build-type> --dependencies <dependencies> --  
node-name <node-name>
```

In ros2 command:

- **pkg**: Package-related functionality;
- **create**: Create a package;
- **package_name**: Required: Name of the new package;
- **build-type**: Required: Indicates whether the new package is C++ or Python. Use `ament_cmake` for C/C++, or `ament_python` for Python;
- **dependencies**: Optional: Package dependencies. C++ packages need `rclcpp`, Python packages need `rclpy`, plus other needed dependencies;
- **node-name**: Optional: Name of the executable program. Automatically generates corresponding source files and configuration files;

For example, creating C++ and Python packages in the terminal:

- Navigate to the `src` directory of the workspace:
- Replace workspace with your actual folder path:

```
cd workspace/src
```

- Create C++ package example:

```
ros2 pkg create pkg_helloworld_cpp --build-type ament_cmake --dependencies rclcpp --node-name helloworld
```

- Create Python package example:

```
ros2 pkg create pkg_helloworld_py --build-type ament_python --dependencies rclpy --node-name helloworld
```

3. Building Packages

After writing code in the created package, you need to build and configure environment variables before running:

- Navigate to the workspace directory:
- Build all packages:

```
colcon build
```

- Build specific packages:

```
colcon build --packages-select pkg1 pkg2
```

4. Complete Workspace Structure with Packages

The ROS2 workspace directory structure is as follows:

```
workspace --- Custom workspace.
  |--- build: Directory storing intermediate files. A separate subdirectory is created for
each package under this directory.
  |--- install: Install directory. A separate subdirectory is created for each package
under this directory.
  |--- log: Log directory for storing log files.
  |--- src: Directory for storing package source code.
    |-- C++ Package
      |-- package.xml: Package information such as name, version, author,
dependencies.
      |-- CMakeLists.txt: Build configuration rules such as source files,
dependencies, and target files.
      |-- src: C++ source file directory.
      |-- include: Header file directory.
      |-- msg: Message interface file directory.
      |-- srv: Service interface file directory.
      |-- action: Action interface file directory.
    |-- Python Package
      |-- package.xml: Package information such as name, version, author,
dependencies.
```

```
|-- setup.py: Similar to CMakeLists.txt for C++ packages.  
|-- setup.cfg: Package basic configuration file.  
|-- resource: Resource directory.  
|-- test: Directory for storing test-related files.  
|-- Package-name directory: Python source file directory.
```

Additionally, both Python and C++ packages can define custom configuration file directories.

```
|-- C++ or Python Package  
  |-- launch: Directory for storing launch files.  
  |-- rviz: Directory for storing rviz2 configuration files.  
  |-- urdf: Directory for storing robot model files.  
  |-- params: Directory for storing parameter files.  
  |-- world: Directory for storing simulation environment files.  
  |-- map: Directory for storing navigation map files.  
  |-- .....
```

These directories can also be given other names, or other directories can be created as needed.